

Process States

Definition

Process states describe the condition a process is in at a given moment, especially whether it is currently executing, waiting for the CPU, or unable to continue until some external event occurs.

1. Why Process States Matter

Main Idea

Although each process has its own program counter and internal state, processes often interact with one another. Because of this interaction, a process may sometimes have to wait instead of continuing immediately.

Pipeline Example

Consider the shell command:

```
cat chapter1 chapter2 chapter3 | grep tree
```

Here:

- **cat** produces output by concatenating files,
- **grep** consumes that output and searches for lines containing **tree**.

If **grep** is ready to run but no input has arrived yet, it cannot continue and must wait.

2. The Three Basic States

Three-State Model

A process can be in one of three basic states:

1. **Running**: the process is currently using the CPU.
2. **Ready**: the process can run, but the CPU is temporarily assigned to another process.
3. **Blocked**: the process cannot run until some external event occurs.

State	Meaning	Can it run immediately?
Running	The process is actively executing.	Yes
Ready	The process is able to run, but is waiting for CPU time.	Yes, when scheduled
Blocked	The process is waiting for input or another external event.	No

3. Ready vs Blocked

Core Distinction

Ready and **blocked** are not the same:

- A **ready** process is willing and able to run, but has no CPU right now.
- A **blocked** process cannot proceed even if the CPU is free.

Why This Difference Matters

The first case is a CPU allocation issue. The second case is a logical waiting issue. For example, a process cannot process keyboard input until the user has actually typed something.

4. The Four State Transitions

Transition Overview

There are four main transitions between the three states.

No.	Transition	Cause
1	Running → Blocked	The process cannot continue and must wait for input or another event.
2	Running → Ready	The scheduler removes the process from the CPU to let another process run.
3	Ready → Running	The scheduler chooses that process to use the CPU.
4	Blocked → Ready	The awaited external event happens, so the process becomes runnable again.

4.1 Transition 1: Running to Blocked

Why It Happens

This transition occurs when the operating system discovers that the process cannot continue right now.

Examples

- In some systems, the process may call a system call such as **pause** to block itself.
- In UNIX, if a process reads from a pipe or a special file like a terminal and no input is available, it is blocked automatically.

4.2 Transitions 2 and 3: Scheduler-Controlled Changes

Scheduler Action

Transitions 2 and 3 are caused by the **process scheduler**, a part of the operating system. These transitions happen without the process controlling them directly.

Running to Ready

Transition 2 happens when the scheduler decides that the running process has used the CPU long enough and should give another process a turn.

Ready to Running

Transition 3 happens when the scheduler selects a ready process and gives it the CPU again.

Scheduling Goal

Scheduling tries to balance two competing goals:

- **efficiency** for the whole system,
- **fairness** for individual processes.

4.3 Transition 4: Blocked to Ready

External Event

Transition 4 happens when the event a process was waiting for finally occurs, such as the arrival of input or completion of an I/O operation.

What Happens Next

Once the event occurs, the process becomes **ready**. If the CPU is free, it may run immediately. Otherwise, it waits in the ready state until the scheduler selects it.

5. Why the Process Model Is Useful

Simpler Way to Think

The process model makes the operating system easier to understand. Instead of focusing on low-level interrupts, we can think in terms of user processes, disk processes, terminal processes, and other logical activities.

Disk Interrupt Example

Suppose a process is waiting for a disk read to finish. While waiting, it is blocked. When the disk interrupt occurs, the waiting process is unblocked and becomes eligible to run again.

Conceptual Benefit

This view turns many hardware events into process-state changes, which is much easier to reason about than tracking every interrupt directly.

6. Process-Structured Operating System View

Layered Picture

In a process-structured operating system, the lowest layer handles:

- interrupts,
- scheduling,
- and the mechanics of starting and stopping processes.

Above that layer sit many ordinary sequential processes.

Role of the Scheduler

In this model, the scheduler is the base layer, and the rest of the operating system is organized more cleanly in process form.

Reality Check

Real operating systems are usually not as neatly structured as this simplified model, but the model is still very useful for understanding how systems behave.

7. Quick Review

Exam-Focused Points

- A process can be in one of three states: running, ready, or blocked.
- Running means using the CPU right now.
- Ready means able to run but waiting for CPU time.
- Blocked means unable to continue until an external event happens.
- Ready and blocked are different: ready is waiting for the CPU, blocked is waiting for an event.
- The scheduler causes the running-to-ready and ready-to-running transitions.
- External events such as input arrival cause blocked processes to become ready again.
- The process model helps explain system behavior in a cleaner way than thinking only in terms of interrupts.